Day 2 Study Notes

Git & Cloud Basics

AI Training Program

Topic	What it covers
1. Git commands & commits	Repository, working folder, staging area, init, status, add, commit, log, diff, push, pull
2. Branching & Pull Requests	Branches, switch, merge, merge conflicts, PR workflow, team collaboration
3. Cloud vs On-premise	Cloud computing, CapEx vs OpEx, IaaS/PaaS/SaaS, public/private/hybrid, regions
4. Subscriptions & Resource Groups	Azure hierarchy, subscriptions, resource groups, resources, tags, RBAC, locks
5. Azure Portal walkthrough	Navigating one subscription and its resources in the portal
6. Revision	Cheat sheet + 50 MCQs with answer key

Activity for the day: create a Git repo and use the taught commands, with one commit per user in each group. The Git sections link back to your own Task 1 (hello.py) so you can recall exactly what you did.

Topic 1 — Git Commands & Commits

1.1 What are Git and GitHub?

Git is a **version control system**: it records every change to your files so you can see who changed what, go back to older versions, and work in a team without overwriting each other. Git runs on your own computer.

GitHub is a website that hosts Git repositories online. It adds collaboration features such as pull requests, code review, issues and access control. Similar services are GitLab, Bitbucket and Azure Repos.

Git	GitHub
Software installed on your computer	Website / cloud service
Tracks versions locally	Stores a remote copy of the repo online
Works offline	Needs internet
Commands: init, add, commit, branch, merge	Features: pull requests, reviews, issues, collaborators

1.2 The three areas of Git

```
Working folder --add--> Staging area --commit--> Local repo --push--> Remote (GitHub)
 (edit files)            (ready to save)           (history)  <--pull--
```

- **Working folder (working directory):** the files you edit.
- **Staging area (index):** the changes you have chosen to include in the next commit.
- **Local repository:** the saved history of commits, stored in the hidden `.git` folder.
- **Remote repository:** the shared online copy, usually named `origin`.

File states

State	Meaning	Seen in git status as
Untracked	New file Git is not tracking yet	Untracked files
Modified	Tracked file changed since the last commit	Changes not staged for commit
Staged	Change added with git add, ready to commit	Changes to be committed
Committed	Saved safely in the local repo	nothing to commit, working tree clean

1.3 Setting up

```
git --version                                  # check Git is installed
git config --global user.name  "Your Name"     # name shown on your commits
git config --global user.email "you@example.com"
git config --list                              # view settings
```

There are two ways to get a repository:

```
git init                                       # turn the current folder into a NEW repo
git clone https://github.com/user/repo.git     # download an EXISTING repo (remote already set)
```

1.4 Everyday commands

Command	What it does
git status	Shows untracked, modified and staged files — run it often
git add hello.py	Stage one file
git add .	Stage all changes in the current folder
git commit -m "message"	Save staged changes as a snapshot with a message
git log / git log --oneline	Show commit history (full / one line each)
git diff	Show unstaged changes line by line
git diff --staged	Show staged changes
git restore file	Discard changes in the working folder
git restore --staged file	Unstage a file (keep the changes)
git rm --cached file	Stop tracking a file but keep it on disk
git remote add origin <url>	Connect the local repo to a GitHub repo
git remote set-url origin <url>	Change the URL of an existing remote
git remote -v	Show remote URLs
git push -u origin main	First push of a branch; sets upstream tracking
git push	Upload new commits
git fetch	Download remote changes without merging
git pull	Download and merge remote changes (fetch + merge)

1.5 Commits

A **commit** is a snapshot of the staged files at one moment. Each commit has a unique ID (a **hash** such as a5231f2), an author, a date, a message, and a link to its parent commit — so the commits form a history.

- Write clear messages in the imperative: *Add hello program*, *Fix login bug*.
- Make small commits that each do one thing — easier to review and undo.
- Commit only working code; never commit passwords or API keys.
- Use a `.gitignore` file to keep files out of Git (e.g. `__pycache__/`, `.env`, `*.db`).

✎ From your practice files

• **Task 1 (hello.py):** git init → git status (untracked) → git add . → git status (staged) → git commit -m "add hello program" (1 file changed, 1 insertion).
• First git push failed with *No configured push destination* because no remote was set.
• You ran git branch -M main, added origin, fixed the mistyped URL with git remote set-url, checked with git remote -v, and pushed with git push -u origin main.
• Next: edit hello.py → status shows **modified** → add → commit → plain git push.

> ⚠ **Remember / common mistakes**
>
> • git commit only saves what is **staged** — run git add first.
> • Commits are local; nothing reaches GitHub until you push.
> • remote origin already exists → use git remote set-url, not add.
> • Create the GitHub repo first and copy its URL exactly (ends in .git).

Topic 2 — Branching & Pull Requests

2.1 Branches

A **branch** is an independent line of work. The main branch (`main`) holds the stable code; you create a new branch for each feature or fix, work there safely, and merge it back when it's ready. A branch is just a lightweight pointer to a commit, so creating one is instant.

```
main:      A --- B --------------- E   (merge)
                  \                /
feature:           C --- D -------
```

```
git branch                       # list branches (* marks the current one)
git branch feature-login         # create a branch
git switch feature-login         # move to it        (older: git checkout feature-login)
git switch -c feature-login      # create AND move    (older: git checkout -b feature-login)
git branch -M main               # rename current branch to main
git branch -d feature-login      # delete a merged branch
git push -u origin feature-login # publish the branch to GitHub
```

2.2 Merging

Merging brings the commits from one branch into another. You switch to the branch that should **receive** the changes, then merge.

```
git switch main
git merge feature-login
```

Merge type	When it happens
Fast-forward	main has no new commits since the branch was made; Git just moves the main pointer forward
Merge commit (three-way)	Both branches have new commits; Git creates a merge commit with two parents
Merge conflict	Both branches changed the same lines of the same file; Git asks you to choose

Resolving a merge conflict

```
<<<<<<< HEAD
print("Hello from main")
=======
print("Hello from feature")
>>>>>>> feature-login
```

- Open the file and decide what the final code should be; delete the <<<<<<<, ======= and >>>>>>> markers.

- Then: `git add file` → `git commit`.
- Prevent conflicts by pulling often and keeping changes small.

2.3 Pull Requests (PRs)

A **pull request** is a GitHub request to merge one branch into another (usually a feature branch into main). It lets teammates **review** the changes, comment, and approve before anything reaches main. (GitLab calls the same thing a *merge request*.)

PR workflow

Step	Action
1	git pull on main to get the latest code
2	git switch -c my-feature — create a branch
3	Edit files → git add → git commit
4	git push -u origin my-feature
5	On GitHub click **Compare & pull request**; base = main, compare = my-feature; add a title and description
6	Reviewers comment, request changes or approve; push more commits to update the PR
7	**Merge pull request** on GitHub, then delete the branch
8	Locally: git switch main → git pull

Fork vs branch: collaborators with write access create branches in the same repo. People without access **fork** (make their own copy of the repo on GitHub), push to the fork, and open a PR back to the original.

2.4 Group activity — one commit per user

For the Day 2 activity, each group shares one repository and every member makes one commit.

```
# Owner: create the repo on GitHub -> Settings -> Collaborators -> add teammates

# Each member:
git clone https://github.com/<owner>/<repo>.git
cd <repo>
git switch -c add-yourname
# create or edit a file, e.g. members/yourname.py
git add .
git commit -m "Add yourname's file"
git push -u origin add-yourname
# open a pull request on GitHub -> teammate reviews -> merge

# Everyone afterwards:
git switch main
git pull
git log --oneline       # see one commit from each member
```

- Working on **separate files** avoids conflicts.
- If you push straight to main and get *rejected (fetch first)*, someone pushed before you: run `git pull`, then push again.
- Check that `user.name` and `user.email` are set so your commit shows your name.

⚠ **Remember / common mistakes**

- Always pull before starting new work.
- Never force-push (git push --force) to a shared branch.
- Merge into the branch you are **on**: switch to main first, then git merge feature.

Topic 3 — Cloud vs On-premise

3.1 What is cloud computing?

Cloud computing means renting computing services — servers, storage, databases, networking, software, AI — over the internet from a provider such as Microsoft Azure, Amazon Web Services (AWS) or Google Cloud, and paying only for what you use.

On-premise (on-prem) means the organization buys, installs and runs its own servers in its own building or data centre.

	On-premise	Cloud
Cost model	CapEx — large upfront purchase of hardware	OpEx — pay-as-you-go running cost
Setup time	Weeks or months to buy and install	Minutes from a portal
Scaling	Buy more hardware; capacity sits idle when not needed	Scale up/down or out/in on demand
Maintenance	Your IT team handles hardware, power, cooling, patches	Provider manages the physical infrastructure
Control	Full control of hardware and data location	Less physical control; choose regions and settings
Security	Entirely your responsibility	Shared responsibility model
Availability	Depends on your own backup sites	Built-in redundancy across zones and regions

3.2 Key cloud benefits

Term	Meaning
Scalability	Add resources to handle more load (scale up = bigger machine; scale out = more machines)
Elasticity	Automatically add and remove resources as demand rises and falls
High availability	Services keep running even if some hardware fails
Pay-as-you-go	Pay only for what you use; stop paying when you delete the resource
Global reach	Deploy close to users in regions around the world
Disaster recovery	Back up and replicate data to other regions

3.3 Service models — IaaS, PaaS, SaaS

Model	You manage	Provider manages	Azure examples
IaaS Infrastructure as a Service	OS, runtime, apps, data	Hardware, network, virtualization	Virtual Machines, Virtual Network
PaaS Platform as a Service	Your app and data	Hardware + OS + runtime	App Service, Azure SQL Database
SaaS Software as a Service	Just use it (your data/settings)	Everything	Microsoft 365, Outlook, Teams

Shared responsibility: the provider always secures the physical data centre; the customer is always responsible for their data, accounts and access. Everything in between shifts from customer to provider as you go from IaaS → PaaS → SaaS.

3.4 Deployment models

Model	Description
Public cloud	Services from a provider shared across many customers (Azure, AWS, GCP)
Private cloud	Cloud environment used by one organization only, on-prem or hosted
Hybrid cloud	Combination of on-prem/private and public cloud connected together
Multi-cloud	Using more than one public cloud provider

3.5 Regions and availability zones

- **Region:** a geographic area with one or more data centres (e.g. Central India, East US, West Europe). Choose a region close to users or required for data rules.
- **Availability zone:** physically separate data centres within a region, with independent power, cooling and network — protects against a data-centre failure.
- **Region pair:** two regions in the same geography paired for disaster recovery.

> ⚠ **Remember / common mistakes**
>
> • CapEx = capital expenditure (upfront); OpEx = operational expenditure (ongoing).
> • Cloud does not mean security is automatic — you still secure your data and access.

Topic 4 — Subscriptions & Resource Groups

4.1 The Azure hierarchy

```
Microsoft Entra tenant (organization identity directory, formerly Azure Active Directory)
 └── Management groups        (optional: group subscriptions to apply policies/access)
      └── Subscriptions        (billing + access boundary)
           └── Resource groups (logical folders)
                └── Resources  (VM, storage account, web app, database ...)
```

Settings such as access (RBAC) and policies applied at a higher level are **inherited** by everything below it.

4.2 Subscription

A **subscription** is an agreement with Microsoft to use Azure services. Every resource belongs to exactly one subscription.

- **Billing boundary:** each subscription gets its own bill and cost reports.
- **Access boundary:** you can give people access to one subscription and not another.
- Has limits (quotas) on how many resources you can create.
- Organizations often use separate subscriptions for Dev, Test and Production, or per department.
- Common types: Free account, Pay-As-You-Go, Azure for Students, Enterprise Agreement.

4.3 Resource group

A **resource group** (RG) is a logical container that holds related resources for one solution — for example a web app, its database and its storage account for one project.

Rule	Detail
Every resource is in exactly one RG	A resource can be moved to another RG, but cannot be in two at once
RGs cannot be nested	No resource group inside another resource group
RG has a location	It stores the group's metadata; resources inside can be in other regions
Shared lifecycle	Deleting an RG deletes **all** resources inside it — handy for cleanup, dangerous by mistake
Management scope	You can apply access (RBAC), policies, tags, locks and view costs at RG level

Example: `rg-libraryapp-dev` containing an App Service, an Azure SQL database and a storage account.

4.4 Resources

A **resource** is any single service you create: Virtual Machine, Storage Account, App Service (web app), Azure SQL Database, Virtual Network, Key Vault, Azure OpenAI, etc. Every resource has a name, type, region, resource group and subscription. **Azure Resource Manager (ARM)** is the service that handles every create/update/delete request, whether from the portal, CLI, PowerShell or code.

4.5 Organizing and protecting resources

Feature	Purpose
Tags	Name : value labels (e.g. env : dev, owner : team-a) for organizing and cost reports
RBAC — Owner	Full access, including giving access to others
RBAC — Contributor	Create and manage resources, but cannot grant access
RBAC — Reader	View only
Resource locks	**CanNotDelete** (Delete lock) — can change but not delete; **ReadOnly** — can only read
Azure Policy	Enforce rules, e.g. only allow certain regions or require tags
Naming convention	Consistent names like rg-<app>-<env>, vm-<app>-01

⚠ **Remember / common mistakes**

• Hierarchy order: Management group → Subscription → Resource group → Resource.
• Subscription = billing + access boundary; Resource group = logical container.
• Use one resource group per project/environment so you can delete it all when finished and stop costs.

Topic 5 — Azure Portal Walkthrough

The **Azure portal** (`portal.azure.com`) is the web interface for creating and managing Azure resources. The same work can be done with the Azure CLI (`az`), PowerShell, or Infrastructure-as-Code templates. Menu names can change slightly over time, but the layout below is the typical walkthrough of one subscription.

5.1 Finding your way around

Area	What it's for
Home / Dashboard	Starting page with recent resources and shortcuts
Search bar (top)	Fastest way to open any service, resource or setting
Portal menu (☰)	All services, favourites, resource groups, all resources
Cloud Shell (>_ icon)	Browser terminal with Azure CLI / PowerShell
Notifications (bell)	Progress of deployments
Directory + subscription filter	Switch tenant and choose which subscriptions you see

5.2 Walkthrough of one subscription

1 — Open Subscriptions

Search “Subscriptions” and select your subscription. The **Overview** shows the subscription name, ID, status, role and a cost summary.

2 — Cost Management

Cost analysis shows spend by service, resource group or tag. **Budgets** send alerts when spend crosses a limit.

3 — Access control (IAM)

See who has which role (Owner, Contributor, Reader) and add role assignments.

4 — Resource groups

Lists every RG in the subscription. Create one with **+ Create**: choose subscription, name and region → Review + create.

5 — Open a resource group

Overview lists its resources with type and location. Side menu: Activity log, IAM, Tags, Locks, Cost analysis, Deployments.

6 — Resources

Open any resource (e.g. a storage account) to see its Overview, settings, monitoring and the resource group / subscription it belongs to.

7 — Usage + quotas / Resource providers

Check limits and which services are registered for the subscription.

8	**Clean up** Delete the resource group when finished so its resources stop costing money.

5.3 Creating a resource — the common pattern

Every **Create** wizard asks for the same basics: **Subscription → Resource group → Name → Region**, then service-specific settings, optional tags, and **Review + create**. ARM validates the request and deploys it; the bell icon shows progress.

```
# Same thing from Cloud Shell (Azure CLI)
az account show                                  # current subscription
az group create --name rg-day2-demo --location centralindia
az group list --output table
az resource list --resource-group rg-day2-demo --output table
az group delete --name rg-day2-demo              # removes the RG and everything in it
```

Quick Revision Cheat Sheet

Topic	Remember
Git areas	Working folder → (add) → Staging → (commit) → Local repo → (push) → Remote
Start	git init (new) / git clone <url> (existing)
Save	git status → git add . → git commit -m "msg" → git push
Remote	git remote add origin <url>; fix with set-url; check with -v
Branch	git switch -c name; git branch; git merge name (from main)
Sync	git pull = fetch + merge; pull before you push
PR	Branch → push → open PR → review → merge → delete branch
Cloud vs on-prem	OpEx vs CapEx; on-demand scaling; provider runs hardware
IaaS / PaaS / SaaS	VM / App Service, SQL DB / Microsoft 365
Deployment	Public, private, hybrid, multi-cloud
Hierarchy	Tenant → Mgmt group → Subscription → Resource group → Resource
Subscription	Billing + access boundary
Resource group	Logical container; one RG per resource; no nesting; delete RG = delete all
RBAC	Owner (all + grant), Contributor (manage), Reader (view)
Locks	CanNotDelete / ReadOnly
Portal	portal.azure.com; search bar; Cloud Shell; Cost Management; IAM

Practice MCQs (1 – 50)

Try every question first, then check the answer key at the end.

— Git commands & commits —

1. Git is best described as a:

A) Version control system
B) Cloud provider
C) Programming language
D) Database

2. Which is TRUE about Git and GitHub?

A) Git runs locally; GitHub hosts repositories online
B) They are the same thing
C) GitHub is required to use Git
D) Git is a website

3. Which command turns the current folder into a new Git repository?

A) git start
B) git new
C) git create
D) git init

4. Which command downloads an existing remote repository to your computer?

A) git clone
B) git copy
C) git pull
D) git fetch

5. A new file that Git is not yet tracking is shown as:

A) Committed
B) Merged
C) Staged
D) Untracked

6. What does git add . do?

A) Deletes untracked files
B) Stages all changes in the current folder
C) Pushes to GitHub
D) Commits all files

7. Which command saves staged changes as a snapshot?

A) git push
B) git save
C) git commit -m "message"
D) git stage

8. Which command shows which files are untracked, modified or staged?

A) git status
B) git show
C) git diff
D) git log

9. Which command shows the commit history in one line per commit?

A) git status -s
B) git history
C) git diff --short
D) git log --oneline

10. You ran git push on a new repo and got 'No configured push destination'. Why?

A) Git is not installed
B) Nothing was committed
C) Wrong branch name
D) No remote repository was added

11. 'error: remote origin already exists' — what should you use to fix a wrong URL?

A) git remote add origin <url>
B) git init again
C) git remote set-url origin <url>
D) git clone <url>

12. Which command lists the remote URLs?

A) git branch -r -url
B) git remote -v
C) git origin
D) git url

13. What does -u do in git push -u origin main?

A) Uploads only untracked files
B) Undoes the last push
C) Sets upstream tracking so later 'git push' works alone
D) Updates Git

14. Where does Git store a repository's history locally?

A) In the staging area
B) On GitHub only
C) In README.md
D) In the .git folder

15. Which file tells Git which files to ignore?

A) .gitconfig
B) .gitignore
C) ignore.txt
D) README.md

16. git pull is equivalent to:

A) git add + git commit
B) git push + git commit
C) git fetch + git merge
D) git clone + git init

— Branching & Pull Requests —

17. A branch in Git is:

A) An independent line of development
B) A commit message
C) A copy of the whole computer
D) A remote server

18. Which command creates a new branch AND switches to it?

A) git merge new
B) git push new
C) git switch -c new
D) git branch new

19. To merge feature into main, you should first:

A) Clone again
B) Switch to feature
C) Switch to main
D) Delete main

20. A merge conflict happens when:

A) A branch has no commits
B) Two branches have different names
C) You push to GitHub
D) Two branches change the same lines of the same file

21. In a conflicted file, the ======= line separates:

A) Branch names
B) Staged from unstaged
C) Two commits' IDs
D) Your version from the incoming version

22. After fixing a merge conflict you should:

A) Delete the repo
B) git init
C) git add the file, then git commit
D) git clone again

23. A Pull Request on GitHub is used to:

A) Propose, review and merge changes from one branch into another
B) Delete a branch
C) Download code
D) Create a new repository

24. In a PR from feature into main, main is the:

A) Remote
B) Compare branch
C) Base branch
D) Fork

25. Someone without write access who wants to contribute usually:

A) Uses git init on GitHub
B) Pushes directly to main
C) Forks the repo and opens a PR
D) Deletes the repo

26. Your push to main is rejected because a teammate pushed first. What do you do?

A) git pull, then git push
B) Delete your commit
C) git push --force
D) git init

— Cloud vs On-premise —

27. Cloud computing means:

A) Running software without electricity
B) Buying servers for your office
C) Storing files only on USB drives
D) Using computing services over the internet and paying for what you use

28. On-premise infrastructure is mainly which cost type?

A) Free
B) CapEx
C) Subscription only
D) OpEx

29. Pay-as-you-go cloud spending is mainly:

A) OpEx
B) Depreciation
C) CapEx
D) Hardware cost

30. Automatically adding and removing resources as demand changes is called:

A) Latency
B) Elasticity
C) Encryption
D) Virtualization

31. Azure Virtual Machines are an example of:

A) SaaS
B) IaaS
C) On-premise
D) PaaS

32. Azure App Service, where you deploy code without managing the OS, is:

A) Private cloud
B) SaaS
C) PaaS
D) IaaS

33. Microsoft 365 (Outlook, Teams) is an example of:

A) SaaS
B) Hybrid cloud
C) IaaS
D) PaaS

34. A company using its own data centre together with Azure is using:

A) Hybrid cloud
B) Multi-region
C) Public cloud
D) SaaS

35. In the shared responsibility model, who is ALWAYS responsible for the customer's data and access?

A) The internet provider
B) The cloud provider
C) Nobody
D) The customer

36. Physically separate data centres within one Azure region are called:

A) Availability zones
B) Subscriptions
C) Tenants
D) Resource groups

— Subscriptions & Resource Groups —

37. What is the correct Azure hierarchy from top to bottom?

A) Subscription → Management group → Resource → Resource group
B) Resource → Resource group → Subscription → Management group
C) Management group → Subscription → Resource group → Resource
D) Resource group → Subscription → Resource → Tenant

38. An Azure subscription is mainly a boundary for:

A) Billing and access
B) File names
C) Programming languages
D) Git branches

39. A resource group is:

A) A logical container for related resources
B) A physical data centre
C) A type of virtual machine
D) A billing account

40. How many resource groups can one resource belong to at a time?

A) None
B) Exactly one
C) Unlimited
D) Two

41. What happens when you delete a resource group?

A) Nothing until you restart
B) All resources inside it are deleted
C) Resources move to another group
D) Only the group name is removed

42. Can a resource group be placed inside another resource group?

A) Yes, up to 3 levels
B) No, resource groups cannot be nested
C) Only in paid subscriptions
D) Only for VMs

43. Resources in a resource group must be in the same region as the group.

A) True
B) False — they can be in different regions
C) Only storage accounts
D) Only in Central India

44. Which RBAC role can manage resources but cannot grant access to others?

A) Owner
B) Reader
C) Guest
D) Contributor

45. Which lock allows changes but prevents deletion?

A) ReadOnly
B) Archive
C) NoAccess
D) CanNotDelete (Delete)

46. Name : value labels such as env : dev attached to resources are called:

A) Policies
B) Locks
C) Branches
D) Tags

— Azure Portal —

47. What is the web address of the Azure portal?

A) cloud.microsoft.net
B) portal.azure.com
C) azure.portal.io
D) azure.github.com

48. Where in the portal do you see who has Owner/Contributor/Reader roles?

A) Cloud Shell
B) Cost analysis
C) Activity log only
D) Access control (IAM)

49. Which portal feature shows spending by service or resource group?

A) Locks
B) Cost Management / Cost analysis
C) Tags
D) IAM

50. Which service processes every create/update/delete request in Azure, whether from the portal or CLI?

A) Azure DevOps
B) Azure Resource Manager (ARM)
C) GitHub Actions
D) Cloud Shell

Answer Key (1 – 50)

Q	Ans	Note	Q	Ans	Note
1	A	It tracks changes to files over time.	26	A	Never force-push to a shared branch.
2	A		27	D	
3	D		28	B	Large upfront hardware purchase.
4	A	clone also sets up origin.	29	A	
5	D		30	B	
6	B		31	B	
7	C		32	C	
8	A		33	A	
9	D		34	A	
10	D	Add one with git remote add origin <url>.	35	D	
11	C		36	A	
12	B		37	C	
13	C		38	A	
14	D		39	A	
15	B		40	B	
16	C		41	B	
17	A		42	B	
18	C	Older equivalent: git checkout -b new.	43	B	The RG location only stores metadata.
19	C	Merge into the branch you are on.	44	D	
20	D		45	D	
21	D		46	D	
22	C		47	B	
23	A		48	D	
24	C		49	B	
25	C		50	B	

Score yourself: 45–50 excellent • 35–44 good, revise weak topics • below 35 re-read the notes and redo the Git task.